UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Department of Computer Science
Faculty of Computing

Assignment 4

(Lab 3: Pseudo-random number: Randomness Test)

Programme	:	Master of Cyber Security
Subject Code	:	MECR1063
Subject Name	:	Cryptographic Engineering
Session-Sem	:	2024/2025-1

Prepared by : 1) Keertennah Devi A/P Ponnambalam (MEC254026)
Section : 01 / 52 (ODL)
Group / Member ID : 2A

Lecturer : Dr. Muhalim Bin Mohamed Amin

Video Link : https://drive.google.com/file/d/1W2r4OCxdlFYaoRPp0yed3_0NCiPDp8vl/view?usp=drive_link

Date : 24 June 2026

Turnitin (%)		Marks / Remarks
Similarity	**AI**	
5	0	

1. Introduction

Cryptographic systems rely heavily on unpredictable random numbers for generating keys, nonces, and initialization vectors. Since computers are deterministic machines, we cannot produce true randomness natively instead, we use Pseudo-Random Number Generators (PRNGs). These algorithms extend a short seed into a long sequence that appears random but is fundamentally reproducible.

The challenge lies in verifying whether the output of a PRNG is statistically acceptable for cryptographic use. If a sequence has too many ones or zeros, it might indicate a biased generator that an attacker could exploit. In this lab, I used the Monobit (Frequency) Test as specified in the FIPS 140-2 standard to validate 20,000-bit streams, repeating the process until I secured three successful streams.

2. Objectives

a) To generate a 20,000-bit binary sequence using a programming language.
b) To implement the FIPS 140-2 Monobit test logic.
c) To iteratively test streams, displaying zeros and ones counts for every attempt.
d) To analyze the statistical properties of the accepted streams.

3. Methodology and Implementation

I used Google Colab with Python for this lab. Instead of the standard random module, I opted for the secrets library. The secrets module uses OS-provided entropy sources like system interrupts and hardware randomness, making it more suitable for cryptographic demonstrations.

Code Logic Steps:

- Generation: secrets.randbits (20000) creates an integer with 20,000 random bits. I convert this to a padded binary string to guarantee exactly 20,000 digits.
- Counting: I sum up the integer values of the bits to count the 1s. The 0s are simply 20000 - ones.
- Test Condition: The FIPS 140-2 standard requires the number of ones, x to satisfy $9725 < x < 10275$ (strict inequality).
- Loop: A while loop runs until a counter for accepted streams reaches 3. Every single iteration prints the counts and result.

4. Results and Analysis

4.1 Output Summary

When I executed the script, I got all three streams were accepted on the very first three attempts. Below is the exact data captured from my Colab output:

Total attempts taken to get 3 successes: 3

Attempt No.	Number of 1s	Number of 0s	Status
1	9948	10052	ACCEPTED
2	10037	9963	ACCEPTED
3	9920	10080	ACCEPTED

4.2 Statistical Analysis

For a truly random 20,000-bit sequence, the expected number of ones is exactly $10{,}000$.

- My accepted counts are $[9948, 10037, 9920]$.
- The average of these three counts is $\frac{9948+10037+9920}{3} = 9968.33$, which is remarkably close to the expected mean of 10,000.
- To understand *why* the FIPS range is set to 9,725–10,275, I calculated the standard deviation for a binomial distribution of this size: $\sigma = \sqrt{n \cdot p \cdot (1-p)} = \sqrt{20000 \cdot 0.5 \cdot 0.5} = \sqrt{5000} \approx 70.71$.
  The acceptable range 9725 to 10275 represents a margin of ± 275 from the mean, which is about ± 3.89 standard deviations. This covers over 99.99% of expected outcomes for a fair generator. Since all three of my counts fall well inside this tight margin, it strongly indicates that the secrets PRNG is producing high-quality, unbiased bits.

5. Discussion

When I first ran the code, I was half-expecting a rejection just to make the report more dramatic. However, getting 3 out of 3 accepted immediately is statistically plausible about a 99.99% chance per stream to fall inside that wide range, so a 100% success rate over 3 attempts isn't suspicious. It just proves the generator is well-seeded and stable.

Even though all my attempts passed, I strictly followed the lab requirement to print the zeros and ones for every cycle. If a rejection had occurred, displaying the exact counts would have immediately shown a bias toward zeros, helping to diagnose whether the seed or algorithm was faulty.

I implemented the test using $9725 < x < 10275$. If I had used <= by mistake, the acceptable window would be wider (allowing 9725 and 10275). This might seem minor, but in a strict FIPS audit, inclusive bounds could pass a stream that is technically non-compliant. Ensuring strict inequality is a critical detail for real-world cryptographic validation.

While my streams passed this frequency test, I recognize that the Monobit test alone is insufficient to guarantee cryptographic security. The FIPS 140-2 suite includes Poker, Runs, and Long Run tests. A biased generator could potentially pass the frequency test but fail the runs test. Therefore, this lab is just the first step in a comprehensive randomness evaluation.

6. Conclusion

In this lab, I successfully generated 20,000-bit streams using Python's secrets library and implemented the FIPS 140-2 Monobit test. The results were clean and decisive all three required streams were accepted without a single rejection, with ones counts hovering comfortably around the ideal 10,000 mark.

This lab reinforced the concept that PRNGs must be rigorously tested against statistical benchmarks before being trusted in cryptographic applications. Although my specific execution ran smoothly, the iterative loop structure I built ensures that even if a future generator produces a fluke biased stream, the system will automatically reject it and try again until it meets the standard.

7. Appendix

```python
import secrets

def generate_20k_bits():
    # Generate cryptographically strong random bits
    bits = bin(secrets.randbits(20000))[2:].zfill(20000)
    return [int(b) for b in bits]

def monobit_test(bits):
    ones = sum(bits)
    zeros = len(bits) - ones
    passed = 9725 < ones < 10275  # Strict inequality per FIPS 140-2
    return passed, ones, zeros

def main():
    accepted = []
    attempts = 0
    max_accepted = 3

    print("=" * 50)
    print("Monobit Test (FIPS 140-2) on 20,000-bit streams")
    print("=" * 50 + "\n")

    while len(accepted) < max_accepted:
        attempts += 1
        bits = generate_20k_bits()
        passed, ones, zeros = monobit_test(bits)

        print(f"Attempt #{attempts}:")
        print(f"  Number of 1s  = {ones}")
        print(f"  Number of 0s  = {zeros}")
        print(f"  Result        = {'ACCEPTED' if passed else 'REJECTED'}\n")

        if passed:
            accepted.append(ones)

    print("-" * 50)
    print(f"Success! Obtained 3 accepted streams after {attempts} attempt(s).")
    print(f"Accepted ones counts: {accepted}")
    print("-" * 50)

if __name__ == "__main__":
    main()
```

Output:

```
==================================================
Monobit Test (FIPS 140-2) on 20,000-bit streams
==================================================

Attempt #1:
  Number of 1s  = 9948
  Number of 0s  = 10052
  Result        = ACCEPTED

Attempt #2:
  Number of 1s  = 10037
  Number of 0s  = 9963
  Result        = ACCEPTED

Attempt #3:
  Number of 1s  = 9920
  Number of 0s  = 10080
  Result        = ACCEPTED

--------------------------------------------------
Success! Obtained 3 accepted streams after 3 attempt(s).
Accepted ones counts: [9948, 10037, 9920]
--------------------------------------------------
```